**UNIVERSITY COLLEGE OF SCIENCE AND TECHNOLOGY**
Department of Engineering Sciences and Applied Arts · Bachelor of Computer Systems Engineering

Edu Bridge
*Bridges Beyond Borders*

**Edu Bridge – Smart Scholarship and Student Services Management System**

# AI Integration & Automation

Matching engine · Groq API · Readiness score · Automation · Evaluation



| | |
|---|---|
| **Prepared by** | **Janeen Osama Atiya Al-Qutati · 220217280**<br>**AI recommendation, CRM & evaluation** |
| **With contributions from** | Lamia Fathi Hamdi Madi (220217142) — Backend development (PHP API)<br>Alaa Suleiman Abdulrahman Abu Mustafa (220217287) — Database design & integration<br>Nada Musa Abdul Qader Al-Asital (220217234) — Frontend development (React.js) |
| **Course supervisor** | Ms. Amina Al-Salout |
| **Academic year** | 2026/2027 |
| **Document version** | 1.0 — October 2026 |

# Document information

This report is a team deliverable of the graduation project. The table below shows which student prepared each part.

**Table 1:** *Contribution of each student*

| Part of this document | Prepared by | Student ID |
|---|---|---|
| AI approach decision, matching algorithm, readiness score, prompt design, guardrails, evaluation plan (§2–§7, §9, §10) | Janeen Osama Atiya Al-Qutati | 220217280 |
| API integration of the Groq client, automation jobs and scheduler (§6.5, §8) | Lamia Fathi Hamdi Madi | 220217142 |
| Data for matching and evaluation, match cache and readiness data (§4.4, §5, §9.2) | Alaa Suleiman Abdulrahman Abu Mustafa | 220217287 |
| Display of recommendations, explanations and readiness in the interface (§3, §11) | Nada Musa Abdul Qader Al-Asital | 220217234 |

**Table 2:** *Document details*

| Item | Value |
|---|---|
| Project | Edu Bridge – Smart Scholarship and Student Services Management System |
| AI approach | Hybrid: weighted rule engine + pre-trained LLM via Groq API (no custom training in v1) |
| Code | `edubridge-api/src/Services/Matching.php` (implemented), `GroqClient.php` (planned) |
| Data source | Real results exported from the MySQL demo database |

> **Key principle**
> The AI component is decision support. It ranks and explains; advisors decide. It never promises admission.

# Table of Contents

Document information ..... 2
1. Introduction ..... 4
2. Approach: Train a Model or Use a Ready API? ..... 5
2.1 Why Groq ..... 5
3. System Design of the AI Component ..... 6
4. Scholarship Matching Algorithm ..... 7
4.1 Criteria and weights ..... 7
4.2 Formula and classification ..... 7
4.3 Worked example (real data) ..... 7
4.4 Results on the demo database ..... 8
4.5 Implementation ..... 8
5. Student Readiness Score ..... 9
6. LLM Integration with the Groq API ..... 10
6.1 What the LLM does — and does not do ..... 10
6.2 Request flow ..... 10
6.3 Configuration ..... 11
6.4 Prompt design ..... 11
6.5 PHP client ..... 11
6.6 Guardrails, privacy and cost ..... 12
7. AI-Assisted Document Checking ..... 13
8. Automation ..... 14
8.1 Life cycle of a recommendation ..... 15
9. Evaluation Plan ..... 16
9.1 Metrics ..... 16
9.2 Test set ..... 17
9.3 Future: training our own model ..... 17
10. Risks and Ethics ..... 18
11. Implementation Status and Plan ..... 19
References ..... 20

*If page numbers do not appear, right-click the table and choose "Update Field".*

# 1. Introduction

This report explains the intelligent part of EduBridge: how the system recommends scholarships, how it measures a student's readiness, how a pre-trained language model (through the **Groq API**) is connected to explain the results, and how routine work is automated with events and scheduled jobs. It also answers the key design question of the project: **should we train our own AI model or use a ready-made one?**

The proposal requires an AI-assisted component that evaluates document completeness, calculates a readiness score and generates scholarship compatibility scores, while acting as **decision support** and never replacing human judgement. Everything in this report follows that principle.

**Table 3:** *Objectives of the AI component*

| Goal | How EduBridge achieves it |
|---|---|
| Recommend suitable scholarships | Weighted rule-based matching (7 criteria) → score 0–100 and a level |
| Explain each recommendation | Groq LLM writes a short Arabic / English explanation of the score |
| Measure file completeness | Readiness score from weighted required documents |
| Reduce manual follow-up | Event-driven jobs + daily scheduler send reminders and re-match students |
| Keep humans in control | Advisor reviews every proposal; no guarantee of admission is ever claimed |

# 2. Approach: Train a Model or Use a Ready API?

Three options were evaluated against the real situation of the project: a new consultancy platform with no historical admission data, a 15-week schedule, a student team and a zero budget.

**Table 4:** *Comparison of AI approaches*

| Criterion | A. Train our own ML model | B. Pre-trained LLM only (API) | C. Hybrid: rules + LLM API (chosen) |
|---|---|---|---|
| Training data needed | Thousands of labelled student–scholarship outcomes (we have none) | None | None (rules come from scholarship criteria) |
| Explainability | Low (black box) | Medium — may invent reasons | High — every point of the score is traceable |
| Consistency | High after training | Low — same input can give different answers | High — score is deterministic |
| Risk of wrong eligibility | Medium | High (hallucination) | Low — LLM never decides eligibility |
| Cost | GPU time + data labelling | Per request | Very low (LLM only for short texts, cached) |
| Time to build | Months | Days | Weeks — fits the plan |
| Arabic + English output | Needs extra work | Yes | Yes |

> **Decision**
>
> **We do not train a model in version 1.** We use a deterministic, weighted rule engine for the score (transparent and testable) and a **ready, pre-trained large language model through the Groq API** only to explain results and to read documents.
>
> Training a dedicated model becomes possible later, after the system has collected real admission outcomes (Section 9).

## 2.1 Why Groq

- **Ready, pre-trained open models** (Llama family) — no training or GPU needed.
- **OpenAI-compatible REST API** — one HTTPS call from PHP with cURL; easy to replace with another provider.
- **Very fast inference**, so an explanation is ready in about one second.
- **Free developer tier** that is enough for the graduation project; usage is limited further by caching.
- Model name is a setting (`GROQ_MODEL` in `.env`), so the team can switch models without code changes.

# 3. System Design of the AI Component

The engine has three layers. Only layer 3 calls the external LLM, and it never changes the score.

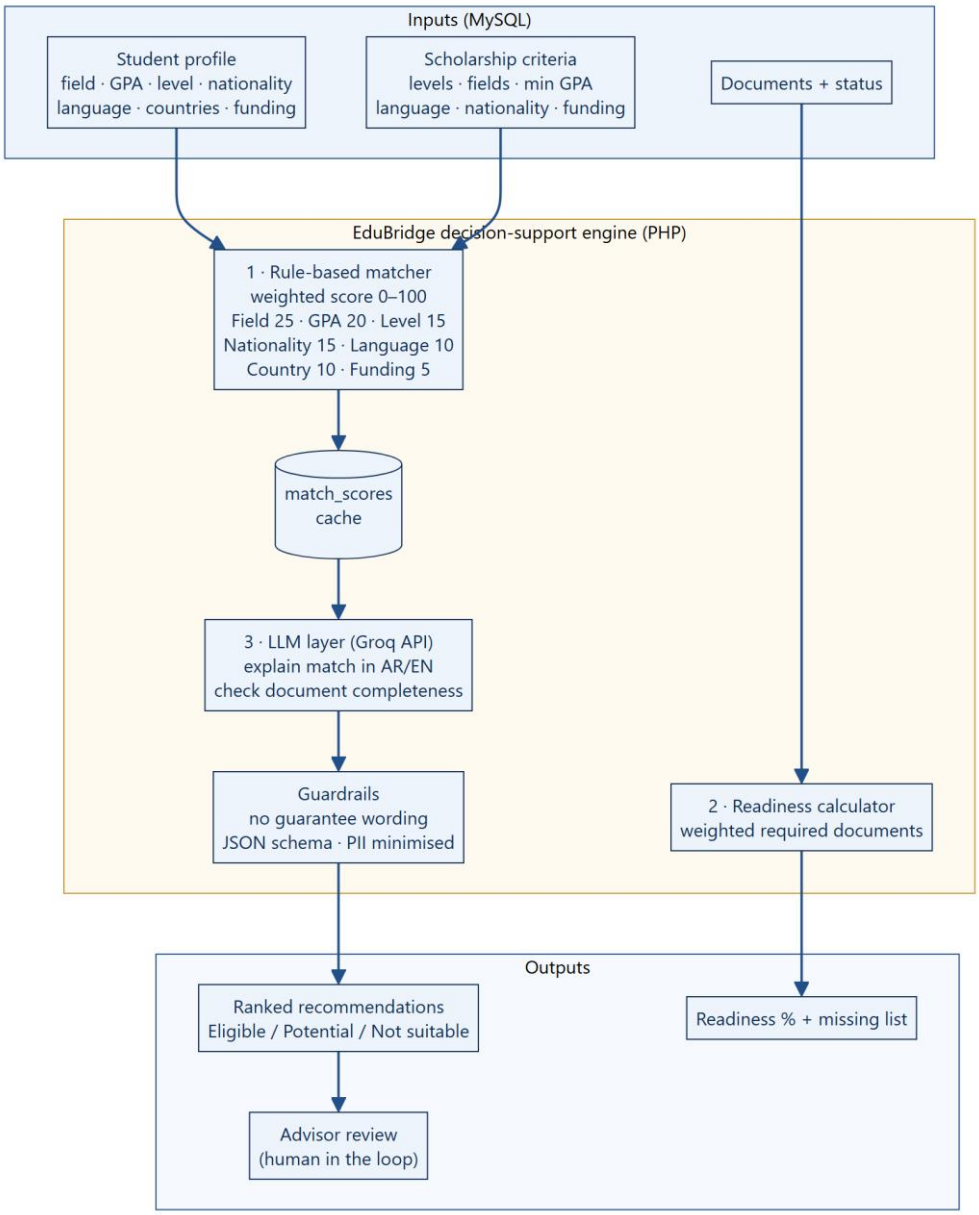



**Figure 1:** *Architecture of the decision-support engine*

**Table 5:** *Layers of the engine*

| Layer | Technology | Output |
|---|---|---|
| 1. Rule-based matcher | PHP `Matching::score()` | Score 0–100, level, per-criterion parts → `match_scores` |
| 2. Readiness calculator | PHP `Matching::readiness()` | Readiness % and missing document list |
| 3. LLM layer | Groq API (chat completions, JSON mode) | Short explanation (AR/EN), gaps, document field extraction |

# 4. Scholarship Matching Algorithm

## 4.1 Criteria and weights

The weights are taken from the project proposal. Each criterion returns a value between 0 and 1 that is multiplied by its weight; the sum is the match score.

**Table 6:** *Matching criteria*

| Criterion | Weight | Full credit (1) | Partial credit | No credit (0) |
|---|---|---|---|---|
| Academic field | 25% | Student field is in the scholarship fields | — | Different field |
| GPA | 20% | GPA ≥ minimum | 0.5 if within 0.3 below the minimum | Lower |
| Study level | 15% | Target level offered | — | Not offered |
| Nationality | 15% | Open to all or to this nationality | — | Not eligible |
| Language | 10% | Score ≥ requirement | 0.5 if within 0.5 below | Lower |
| Preferred country | 10% | Country in the student's list | — | Not preferred |
| Funding | 5% | Same funding type | 0.4 otherwise | — |

## 4.2 Formula and classification

**Score formula**

```
score = Σ ( weight_i × part_i )          part_i ∈ {0, 0.4, 0.5, 1}

level = Eligible            if score ≥ 75
        Potentially suitable if 50 ≤ score < 75
        Not suitable         if score < 50
```

## 4.3 Worked example (real data)

**Sarah Ahmed** (master, computerScience, GPA 3.7, IELTS 6.5) compared with **Global Excellence Scholarship** (University of Toronto, Canada):

**Table 7:** *Calculation for Sarah Ahmed × Global Excellence Scholarship → eligible*

| Criterion | Student | Scholarship | Part | Weight | Points |
|---|---|---|---|---|---|
| Academic field | computerScience | business, computerScience, engineering | 1 | 25 | 25 |
| GPA | 3.7 | ≥ 3.3 | 1 | 20 | 20 |
| Study level | master | master, phd | 1 | 15 | 15 |
| Nationality | palestine | all | 1 | 15 | 15 |
| Language | IELTS 6.5 | IELTS ≥ 7 | 0.5 | 10 | 5 |
| Preferred country | canada, germany | canada | 1 | 10 | 10 |
| Funding | full | full | 1 | 5 | 5 |
| **Total** | | | | 100 | **95** |

The only lost points come from the language criterion: IELTS 6.5 is 0.5 below the required 7.0, so the student receives half of the 10 points. The explanation layer turns this into advice: “Improve your IELTS to 7.0 to reach a full match.”

## 4.4 Results on the demo database

The matrix was produced by running the real `Matching` class over all students and scholarships in the MySQL database (E = Eligible, P = Potentially suitable, N = Not suitable).

**Table 8:** *Match scores (students × scholarships)*

| Student | Global Excellence (CA) | DAAD (DE) | Commonwealth (UK) | Erasmus Mundus (NL) | Türkiye Burslari (TR) | Malaysia Intl. (MY) |
|---|---|---|---|---|---|---|
| Sarah Ahmed | 95 E | 100 E | 85 E | 85 E | 90 E | 62 P |
| Yousef Karimi | 100 E | 100 E | 50 P | 75 E | 90 E | 87 E |
| Noor Hassan | 82 E | 62 P | 92 E | 47 N | 87 E | 100 E |
| Omar Saleh | 30 N | 65 P | 30 N | 45 N | 90 E | 62 P |
| Layla Saeed | 65 P | 65 P | 100 E | 65 P | 65 P | 62 P |
| Hadi Tamimi | 100 E | 90 E | 90 E | 90 E | 90 E | 62 P |

## 4.5 Implementation

**src/Services/Matching.php (simplified)**

```
public const WEIGHTS = ['field' => 25, 'gpa' => 20, 'level' => 15, 'nationality' => 15,
                        'language' => 10, 'country' => 10, 'funding' => 5];

public static function score(array $student, array $s): array {
    $parts = [
        'field'    => in_array($student['field'], $s['fields'], true) ? 1 : 0,
        'gpa'      => $gpa >= $s['minGpa'] ? 1 : ($gpa >= $s['minGpa'] - 0.3 ? 0.5 : 0),
        'level'    => in_array($student['level'], $s['levels'], true) ? 1 : 0,
        // … nationality, language, country, funding
    ];
    $score = round(Σ WEIGHTS[k] × $parts[k]);
    $level = $score >= 75 ? 'eligible' : ($score >= 50 ? 'potential' : 'notSuitable');
    return ['score' => $score, 'level' => $level, 'parts' => $parts];
}
```

# 5. Student Readiness Score

Readiness measures how complete the student's file is. Each required document type has a weight (total 100). An approved document gives its full weight, a document under review 80 % of it, and a missing or rejected document 0.

**Table 9:** *Readiness calculation (real data)*

| Document | Weight | Sarah's status | Points |
|---|---|---|---|
| Passport | 15 | Approved | 15 |
| Academic transcript | 20 | Approved | 20 |
| Degree certificate | 15 | Under review | 12 |
| CV / Résumé | 15 | Missing | 0 |
| Language certificate | 15 | Missing | 0 |
| Motivation letter | 10 | Approved | 10 |
| Recommendation letter | 10 | Under review | 8 |
| **Readiness** | **100** | | **65 %** |

The API returns `{ "score": 65, "missing": ["cv", "language"] }`, which the dashboard shows as a ring and a list of "Upload" buttons.

# 6. LLM Integration with the Groq API

## 6.1 What the LLM does — and does not do

**Table 10:** *Role of the language model*

| The LLM does | The LLM does not |
|---|---|
| Explain a match score in simple Arabic and English | Change or compute the score |
| List the gaps (e.g. “IELTS 6.5 < 7.0”) as advice | Decide eligibility or admission |
| Read a certificate and suggest values (test, score, date) | Approve documents — the advisor does |
| Draft a short note for the advisor | Send anything to the student without review |

## 6.2 Request flow

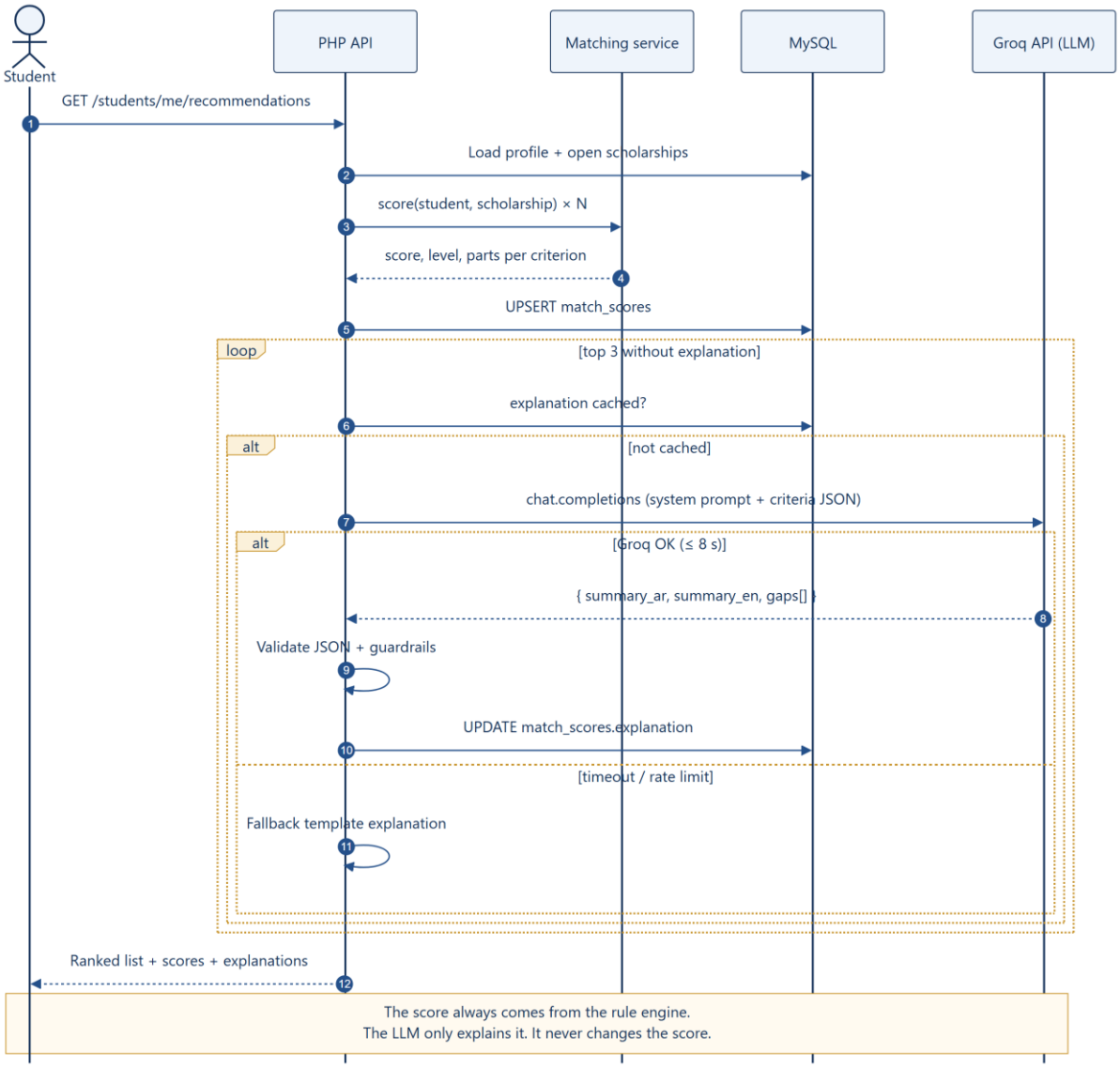



**Figure 2:** *Sequence — recommendations with Groq explanations, cache and fallback*

## 6.3 Configuration

**Settings**

```
# edubridge-api/.env
GROQ_API_KEY=gsk_xxxxxxxxxxxxxxxxxxxx      # from console.groq.com (never commit this file)
GROQ_MODEL=llama-3.3-70b-versatile          # any chat model available on Groq
GROQ_TIMEOUT=8                               # seconds; after that the fallback text is used
```

## 6.4 Prompt design

The prompt sends only the criteria-level comparison — never the student's name, passport number, phone or email — and asks for a strict JSON answer.

**Prompt (system + user message)**

```
SYSTEM:
You are an assistant for EduBridge, an educational consultancy. You explain a
scholarship match score that was ALREADY calculated by rules. Never change the score,
never promise admission, never invent requirements. Answer ONLY with JSON:
{"summary_en": string(<=60 words), "summary_ar": string(<=60 words),
 "strengths": string[], "gaps": [{"criterion": string, "advice_en": string, "advice_ar": string}]}

USER:
{"scholarship": "Global Excellence Scholarship", "score": 95, "level": "eligible",
 "criteria": [
   {"name": "field",    "student": "computerScience", "required":
["computerScience","engineering","business"], "part": 1},
   {"name": "gpa",      "student": 3.7, "required": ">= 3.3", "part": 1},
   {"name": "language", "student": "IELTS 6.5", "required": "IELTS >= 7.0", "part": 0.5}, …]}
```

**Expected response**

```
{
  "summary_en": "A strong match: your field, GPA and study level fit this scholarship well.",
  "summary_ar": "تطابق قوي: تخصصك ومعدلك والمرحلة الدراسية تناسب هذه المنحة.",
  "strengths": ["Computer Science is an eligible field", "GPA 3.7 is above 3.3"],
  "gaps": [{ "criterion": "language",
             "advice_en": "Raise your IELTS from 6.5 to 7.0 to reach a full match.",
             "advice_ar": "ارفعي درجة IELTS من 6.5 إلى 7.0 للوصول إلى تطابق كامل." }]
}
```

## 6.5 PHP client

**src/Services/GroqClient.php (planned)**

```
final class GroqClient {
    public static function explain(array $payload): ?array {
        $ch = curl_init('https://api.groq.com/openai/v1/chat/completions');
        curl_setopt_array($ch, [
            CURLOPT_POST => true, CURLOPT_RETURNTRANSFER => true,
            CURLOPT_TIMEOUT => (int) Env::get('GROQ_TIMEOUT', '8'),
            CURLOPT_HTTPHEADER => ['Content-Type: application/json',
                                   'Authorization: Bearer ' . Env::get('GROQ_API_KEY')],
            CURLOPT_POSTFIELDS => json_encode([
                'model' => Env::get('GROQ_MODEL'),
                'temperature' => 0.2,
                'response_format' => ['type' => 'json_object'],
                'messages' => [['role' => 'system', 'content' => self::SYSTEM_PROMPT],
                               ['role' => 'user', 'content' => json_encode($payload)]],
            ]),
        ]);
        $body = curl_exec($ch);
        $status = curl_getinfo($ch, CURLINFO_HTTP_CODE);
        curl_close($ch);
        if ($status !== 200 || !$body) return null;           // → fallback text
        $json = json_decode(json_decode($body, true)['choices'][0]['message']['content'] ?? '',
true);
```

```
        return Guardrails::check($json) ? $json : null;
    }
}
```

## 6.6 Guardrails, privacy and cost

- **Schema check** — the answer must be valid JSON with the expected keys; otherwise it is discarded.
- **Wording filter** — answers that promise admission (for example “guaranteed” or “100% accepted”, in English or Arabic) are rejected.
- **No personal data** — only criteria values are sent; names and documents are never sent for explanations.
- **Low temperature (0.2)** — stable, factual wording.
- **Cache** — the explanation is stored in `match_scores.explanation` and reused until the profile or scholarship changes, so each pair costs at most one call.
- **Fallback** — if Groq is slow or unavailable, a template explanation is built from the `parts` values, so the page always works.
- **Human in the loop** — advisors see the score, the parts and the explanation before proposing a scholarship.

# 7. AI-Assisted Document Checking

Advisors spend much time opening certificates to read scores and dates. The LLM reads the text of the uploaded PDF and **pre-fills** the review form; the advisor confirms or corrects it.

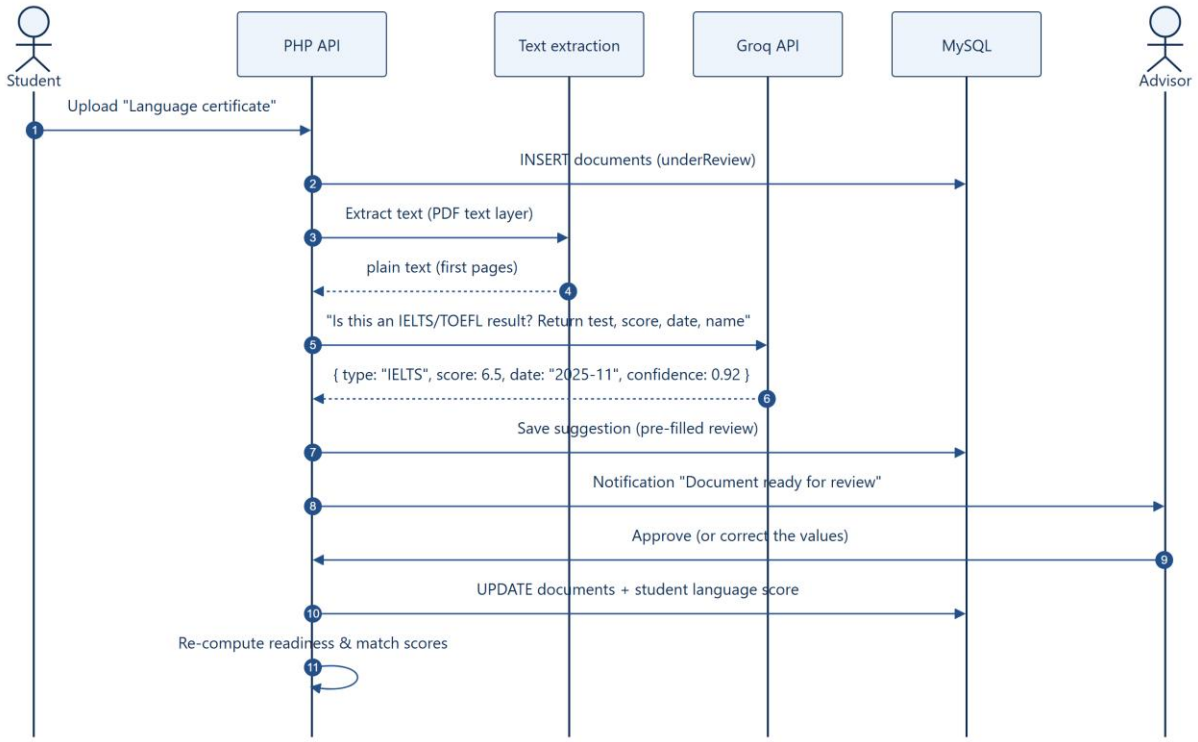



**Figure 3:** *Sequence — document checking with pre-filled review*

**Table 11:** *Document fields*

| Document | Fields suggested by the LLM | Used for |
|---|---|---|
| Language certificate | Test (IELTS/TOEFL), overall score, test date, name | Language criterion of matching |
| Transcript | GPA, scale, institution, graduation year | GPA criterion |
| Passport | Expiry date only (number is not stored) | Warning if it expires within 6 months |

# 8. Automation

Automation removes repetitive follow-up — the main problem of the WhatsApp-based process. Two mechanisms are used: **events** (something changes in the system) and a **daily scheduler** (time-based checks).

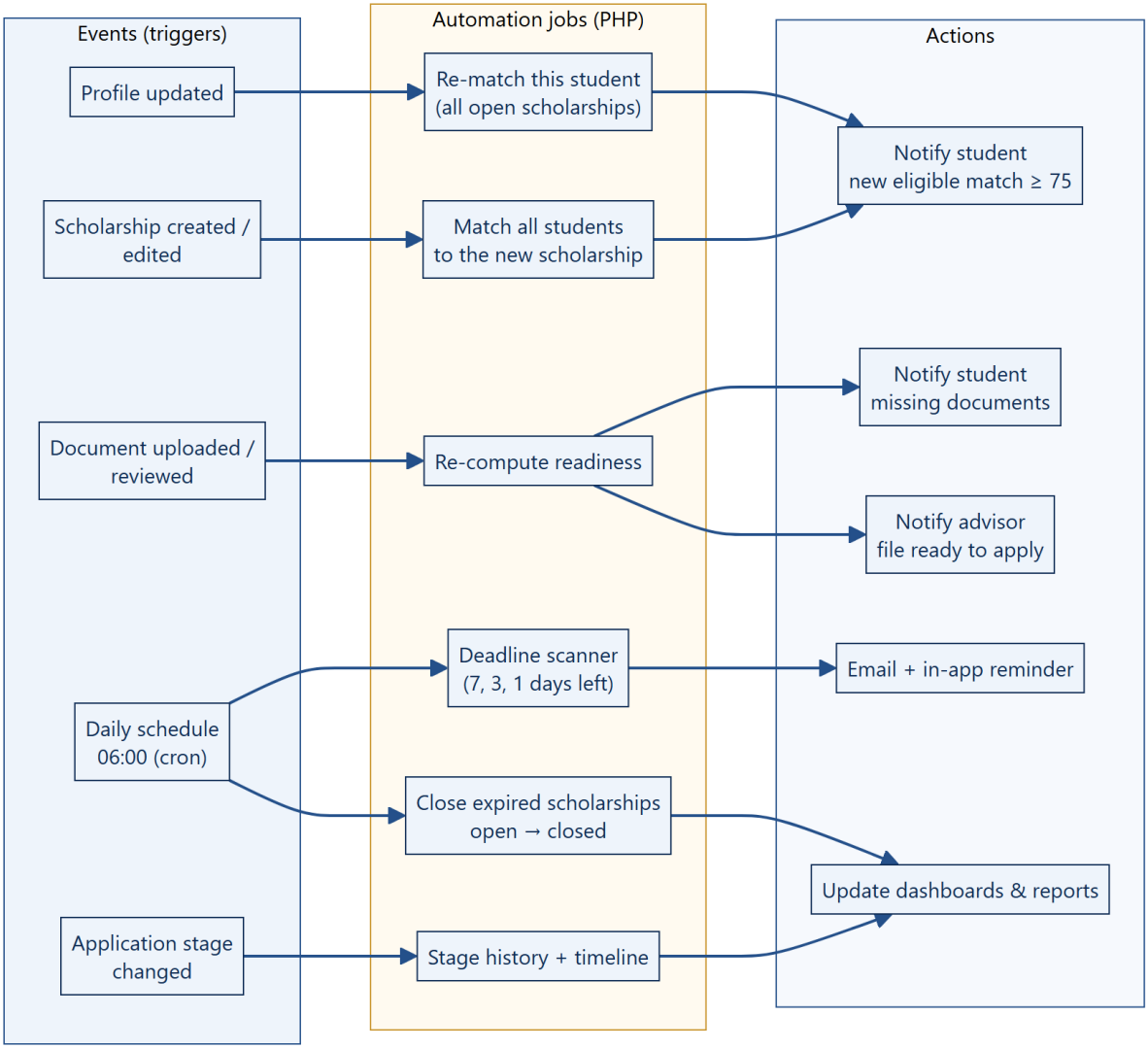



**Figure 4:** *Events, automation jobs and actions*

**Table 12:** *Automation rules*

| Trigger | Job | Result |
|---|---|---|
| Student updates profile | Re-match this student with all open scholarships | New eligible scholarship (≥ 75) → notification |
| Document uploaded or reviewed | Re-compute readiness | Missing list updated; advisor told when the file is complete |
| Scholarship created / edited | Match every student to it | Eligible students notified |
| Daily 06:00 | Deadline scanner (7, 3, 1 days left) | Email + in-app reminders |
| Daily 06:00 | Close scholarships after the deadline | Status open → closed; reports updated |
| Stage changed | Write stage history | Timeline and statistics updated |

### Scheduling the daily job

**Scheduler**

```
# Linux server (crontab -e)
0 6 * * *  php /var/www/edubridge-api/bin/daily.php >> /var/log/edubridge-daily.log 2>&1

:: Windows / XAMPP (Task Scheduler, run as administrator)
schtasks /Create /SC DAILY /ST 06:00 /TN "EduBridge daily" ^
         /TR "php D:\edubridge-frontend\edubridge-api\bin\daily.php"
```

Jobs are **idempotent**: a reminder is recorded in `notifications`, and the job checks this table before sending, so running a job twice never sends duplicate messages.

## 8.1 Life cycle of a recommendation

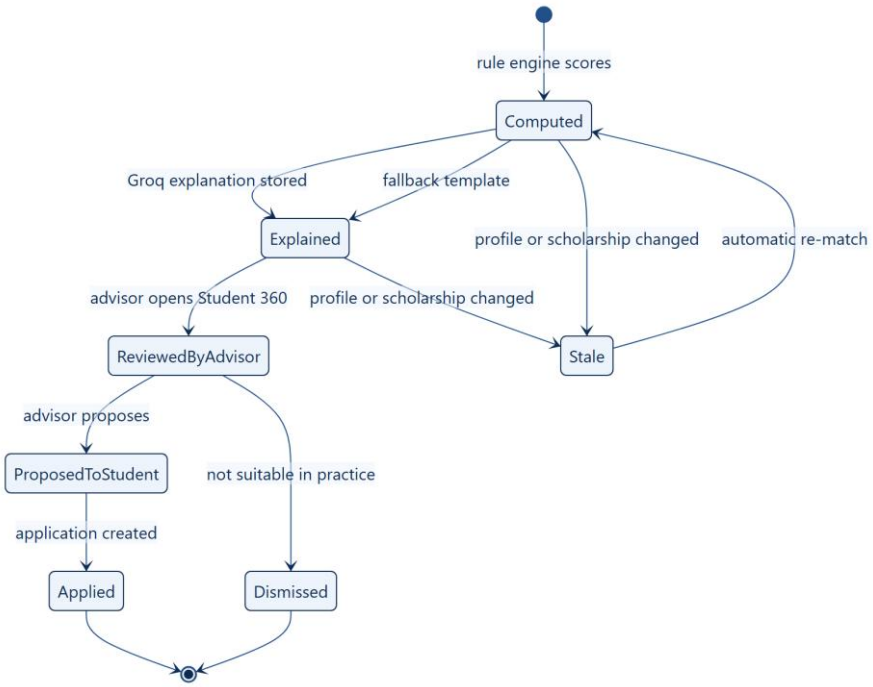



**Figure 5:** *State diagram — life cycle of a match result*

# 9. Evaluation Plan

The proposal asks to measure matching accuracy, false recommendations, missed suitable scholarships, explanation quality and consistency. The evaluation follows the activity diagram below.

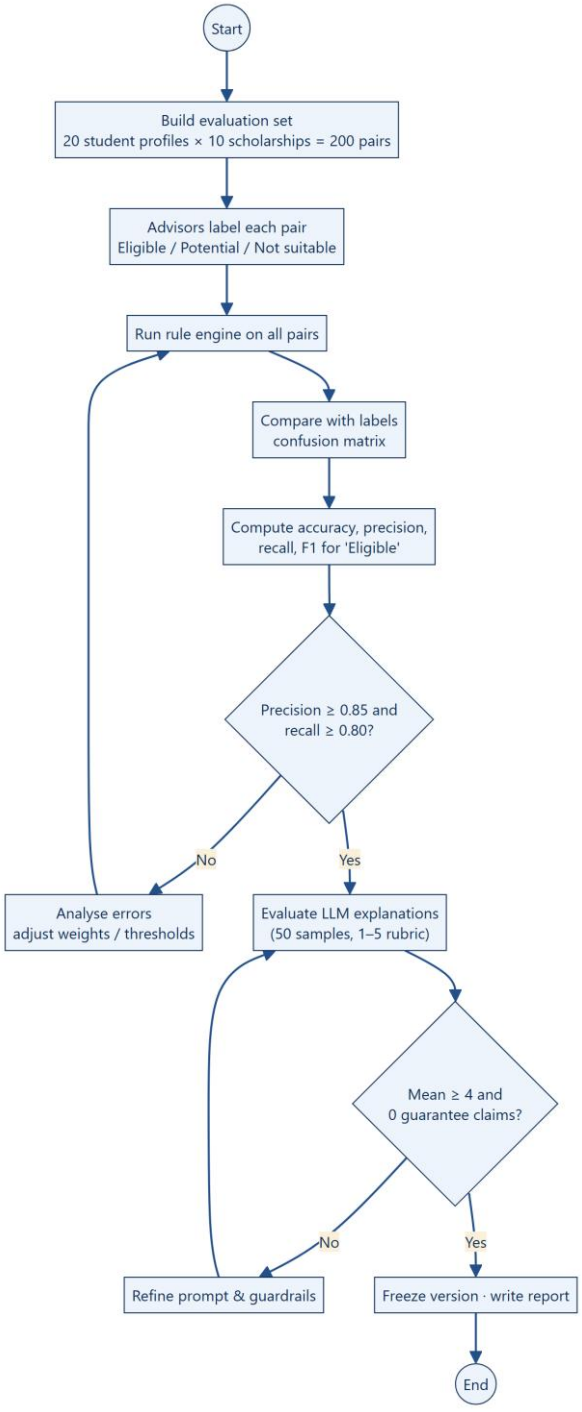



**Figure 6:** *Activity diagram — evaluation process*

## 9.1 Metrics

**Table 13:** *Evaluation metrics*

| Metric | Formula | Target |
|---|---|---|
| Accuracy | (correct levels) ÷ (all pairs) | ≥ 0.85 |

| Metric | Formula | Target |
|---|---|---|
| Precision (Eligible) | TP ÷ (TP + FP) — how many "Eligible" were right | ≥ 0.85 |
| Recall (Eligible) | TP ÷ (TP + FN) — how many truly suitable we found | ≥ 0.80 |
| False recommendation rate | FP ÷ (FP + TN) | ≤ 0.10 |
| Consistency | Same input twice → same score | 100 % (deterministic) |
| Explanation quality | Advisor rating 1–5 (clarity, correctness, usefulness) | Mean ≥ 4 |
| Safety | Explanations claiming guaranteed admission | 0 |

## 9.2 Test set

- 20 anonymised student profiles × 10 scholarships = **200 pairs**.
- Two advisors label each pair independently; disagreements are discussed and resolved.
- The same set is kept as a regression test: after any change to weights or thresholds the metrics are recomputed.

**Table 14:** *Confusion matrix template*

| | Advisor: Eligible | Advisor: not Eligible |
|---|---|---|
| Engine: Eligible | TP | FP (false recommendation) |
| Engine: not Eligible | FN (missed scholarship) | TN |

## 9.3 Future: training our own model

When the platform has collected enough real outcomes (accepted / rejected decisions stored in `applications.result`), the team can train a supervised model (e.g. logistic regression or gradient boosting) on the same seven features plus history. It will be adopted only if it beats the rule engine on the frozen test set while staying explainable. Until then the rule engine is the safer and more transparent choice.

# 10. Risks and Ethics

**Table 15:** *Risks and mitigations*

| Risk | Mitigation |
|---|---|
| Student believes a high score means admission | Clear disclaimer; wording filter; level labels instead of promises |
| LLM invents requirements | LLM never decides; JSON schema; only rule-engine facts are given to it |
| Personal data leaves the system | Only criteria values are sent; no names, IDs or documents for explanations |
| Outdated scholarship data → wrong matches | Staff own the data; automatic re-match on every edit; deadline job closes old items |
| External API down or rate-limited | Timeout, cache and template fallback |
| Bias in weights | Weights are documented, reviewed with Edu Bridge staff and tested on the labelled set |

# 11. Implementation Status and Plan

**Table 16:** *Status of the AI and automation work*

| Item | Status | Owner |
|---|---|---|
| Rule-based matching engine (PHP) + `match_scores` cache | Done — running in the API | Janeen |
| Readiness score + missing list | Done — running in the API | Janeen, Alaa |
| Recommendations & readiness endpoints | Done — tested in Swagger / Postman | Lamia |
| Dashboard display (ring, ranked cards, match levels) | Done — React | Nada |
| Groq client, prompt, guardrails, fallback | Planned — week 3 | Janeen, Lamia |
| Document field extraction | Planned — week 3 | Janeen |
| Daily scheduler and event jobs | Planned — week 3 | Lamia |
| Evaluation set (200 pairs) and report | Planned — week 4 | Janeen, Alaa |

**Summary**

EduBridge does **not** train its own model in version 1. It combines a transparent **weighted rule engine** (the score) with a **ready, pre-trained LLM through the Groq API** (the explanation and document reading), plus **event and scheduled automation** for reminders and re-matching.

This gives accurate, explainable, Arabic/English decision support at almost no cost, and keeps the advisor in control of every final decision.

# References

1. Edu Bridge project proposal, University College of Science and Technology, 2025/2026.
2. Groq. GroqCloud API documentation — OpenAI-compatible chat completions. https://console.groq.com/docs
3. Meta. Llama models — model cards and responsible use guide.
4. scikit-learn developers. Model evaluation: precision, recall and F1 score.
5. OWASP. Top 10 for Large Language Model Applications.